



Atelier Complet Full Stack-BackEnd 1

Khalid Nafil
ENSIAS

Dans ce Lab, nous considérons la mise en place d'un projet Spring Boot qui utilise les web services REST. Il s'agit d'un magasin en ligne de ventes de Voitures. La base de données contient deux entités : Voiture et Propriétaire. Ces deux entités sont reliées par une association de type Un à Plusieurs. Nous utilisons l'IDE IntelliJ Idea pour la création du projet Spring Boot. Pour le reste du Lab, vous allez suivre les étapes suivantes.

1. Créez un projet Spring Boot Starter sous IntelliJ Idea sous le nom 'SpringDataRest'.
2. Choisir les dépendances **Web** et **Devtools** pour commencer. //Veuillez se renseigner par rapport à la dépendance Devtools
3. Pour cet atelier, veuillez installer le serveur de base de données MariaDB <https://downloads.mariadb.org/>, ou bien Postgresql.
4. Faites : **sudo mysql -u root -p** et vous tapez le mot de passe root.
5. Une fois connectée à Mariadb et créé la base de données 'springboot' faites : **use springboot** (Entrée) et après faites **select * from Voiture;** pour afficher la liste des voitures déjà existantes dans la base.

6. Nous allons commencer par utiliser d'abord la base de données intégrée (in-memory database) à Spring Boot qui est la base H2 en mode développement ou test.
7. Pour pouvoir utiliser **H2** et **JPA** vous devez les ajouter en faisant un bouton droit sur le nom du projet et choisir Spring Starter et puis cochez ces deux dépendances, elles vont être ajoutées au fichier pom.xml.
8. Ajoutez aussi la dépendance **Lombok**.
9. Vous créez le fichier de propriétés 'application.properties' qui contient les informations suivantes :

```
spring.h2.console.enabled=true  
  
spring.datasource.platform=h2  
  
spring.datasource.url=jdbc:h2:mem:testbd
```

10. Créez la classe Voiture dans le package 'modele' du root package 'org.cours'.

```
package org.cours.modele;  
  
...  
  
import lombok.Data;  
  
import lombok.NoArgsConstructor;  
  
import lombok.NonNull;  
  
import lombok.RequiredArgsConstructor;  
  
@Entity  
@Data  
@RequiredArgsConstructor  
@NoArgsConstructor  
  
public class Voiture {  
  
    @Id  
  
    @GeneratedValue(strategy=GenerationType.AUTO)  
  
    private long id;
```

```

        @NonNull

        private String marque;

        @NonNull

        private String modele;

        @NonNull

        private String couleur;

        @NonNull

        private String immatricule;

        @NonNull

        private int annee;

        @NonNull

        private int prix;

    }

```

11. Créez maintenant l'interface 'VoitureRepo' qui va étendre CrudRepository dans le package 'org.cours.modele'.

12. CrudRepository fournit plusieurs méthodes dont ci-après la liste de celles parmi les plus utilisées :

- a. long count(), retourne le nombre d'entités;
- b. Iterable<T> findAll(), retourne tous les items d'un certain type;
- c. Optional<T> findById(ID id), retourne un item par id;
- d. void delete(T entity), supprime une entité;
- e. void deleteAll(), supprime toutes les entités de cette repository;
- f. <S extends T> save(S entity), sauvegarde une entité. Si la méthode retourne un seul élément, Optional<T> est retournée au lieu de T. Optional est un conteneur d'une seule valeur qui soit possède une valeur soit ne possède aucune. En utilisant Optional, on peut prévoir des exceptions de pointeur null.

```
package org.cours.modele;
```

```
public interface VoitureRepo extends CrudRepository<Voiture, Long> {
```

```
}
```

13. Nous allons après ajouter des données de test à notre base de données H2 en utilisant l'interface **CommandLineRunner** de Spring Boot, qui va nous permettre d'exécuter un code additionnel avant que l'application ne puisse démarrer complètement.

14. `@SpringBootApplication`

```
public class SpringDataRest {  
  
    @Autowired  
  
    private VoitureRepo repository;  
  
    public static void main(String[] args) {  
  
        SpringApplication.run(SpringDataRest.class, args);  
  
    }  
  
    @Bean  
  
    CommandLineRunner runner(){  
  
        return args -> {  
  
            repository.save(new Voiture("Toyota", "Corolla", "Grise", "A-1-9090", 2018,  
95000));  
  
            repository.save(new Voiture("Ford", "Fiesta", "Rouge", "A-2-8090", 2015, 90000));  
  
            repository.save(new Voiture("Honda", "CRV", "Bleu", "A-3-7090", 2016, 140000));  
  
        };  
  
    }  
  
}
```

15. Lancez l'exécution du projet et via la console de H2, vous pouvez consulter les voitures ajoutées à la base (<http://localhost:8080/h2-console>).

16. Pour les opérations de manipulation des éléments de l'entité Voiture, vous pouvez via l'interface **VoitureRepo** définir vos propres requêtes comme suit (à titre indicatif, pour le moment n'ajouter aucune méthode) :

```

package org.cours.modele;

public interface VoitureRepo extends CrudRepository<Voiture, Long> {

    // Sélectionnez les voitures par marque
    List<Voiture> findByMarque(String marque);

    // Sélectionnez les voitures par couleur
    List<Voiture> findByCouleur(String couleur);

    // Sélectionnez les voitures par année
    List<Voiture> findByAnnee(int annee);

    // Sélectionnez les voitures par marque et modele
    List<Voiture> findByMarqueAndModele(String marque, String modele);

    // Sélectionnez les voitures par marque ou couleur
    List<Voiture> findByMarqueOrCouleur(String marque, String couleur);

    // Sélectionnez les voitures par marque et trier par annee
    List<Voiture> findByMarqueOrderByAnneeAsc(String marque);

    // Sélectionnez les voitures par marque en utilisant SQL
    @Query("select v from Voiture v where v.marque = ?1")
    List<Voiture> findByMarque(String marque);

    @Query("select v from Voiture v where v.marque like %?1")
    List<Voiture> findByMarqueEndsWith(String marque);

}

```

17. Nous créons maintenant l'entité **Proprietaire** au niveau du package 'modele' :

...

```

import lombok.Data;

import lombok.NoArgsConstructor;

```

```

import lombok.NonNull;

import lombok.RequiredArgsConstructor;

@Entity
@Data
@RequiredArgsConstructor
@NoArgsConstructor

public class Proprietaire {

    @Id

    @GeneratedValue(strategy=GenerationType.AUTO)

    private long id;

    @NonNull

    private String nom;

    @NonNull

    private String prenom;

}

```

18. Nous créons après l'interface **ProprietaireRepo** comme suit :

```

package org.cours.modele;

public interface ProprietaireRepo extends CrudRepository<Proprietaire, Long> {

}

```

19. Exécutez pour voir si tout marche bien.

20. Maintenant on va relier les deux entités **Voiture** et **Proprietaire** par une association de type **ManyToOne** côté Voiture. Pour cela nous allons ajouter un attribut **proprietaire** au niveau de la classe Voiture et l'annoter par l'annotation **@ManyToOne** comme suit :

```

@ManyToOne(fetch = FetchType.LAZY)

@JoinColumn(name = "proprietaire")

```

@NonNull

private Proprietaire proprietaire;

21. Dans la classe **Proprietaire**, on apporte les modifications suivantes :

@OneToMany(cascade = CascadeType.ALL, mappedBy="proprietaire")

private List<Voiture> voitures;

22. Exécutez pour voir les nouveaux changements.

23. Nous allons ajouter à ce niveau deux propriétaires comme pour les voitures au niveau de CommandLineRunner, après avoir ajouté : **@Autowired ProprietaireRepo proprietaireRepo** :

Proprietaire proprietaire1 = new Proprietaire("Ali", "Hassan");

Proprietaire proprietaire2 = new Proprietaire("Najat", "Bani");

proprietaireRepo.save(proprietaire1);

proprietaireRepo.save(proprietaire2);

repository.save(new Voiture("Toyota", "Corolla", "Grise", "A-1-9090", 2018, 95000), proprietaire1);

repository.save(new Voiture("Ford", "Fiesta", "Rouge", "A-2-8090", 2015, 90000, proprietaire1));

repository.save(new Voiture("Honda", "CRV", "Bleu", "A-3-7090", 2016, 140000, proprietaire2));

24. Pour passer après sur *MariaDB*, il faut chercher sa dépendance sur le site *mvnrepository* et la placer sur le fichier pom.xml. Pour ceux qui utilisent *Postgresql*, essayez d'ajouter sa dépendance depuis Spring Boot.

25. Au niveau du fichier **application.properties** vous allez placer les lignes suivantes (où compagnie c'est le nom de la BD) et enlever ceux de H2 :

spring.datasource.url=jdbc:mariadb://localhost:3306/compagnie

spring.datasource.username=root

spring.datasource.password=YOUR_PASSWORD

spring.datasource.driver-class-name=org.mariadb.jdbc.Driver

```
spring.jpa.generate-ddl=true

spring.jpa.hibernate.ddl-auto=create-drop

//pour Postgresql

spring.datasource.url=jdbc:postgresql://localhost:5432/springboot

spring.datasource.username=postgres

spring.datasource.password=admin

spring.datasource.driver-class-name=org.postgresql.Driver

spring.jpa.generate-ddl=true

spring.jpa.hibernate.ddl-auto=create-drop

//pour générer le path /api

spring.data.rest.base-path=/api
```

26. Vous pouvez exécuter le projet et voir les tables créées au niveau de MariaDB ou Postgresql.

Spring Rest

1. A ce stade, nous allons créer un webservice RESTful. Nous commençons par créer la classe VoitureController dans le package 'org.cours.web'.
2. Cette classe va être annotée par @RestController pour indiquer qu'elle va jouer le rôle du contrôleur du webservice RESTfull.
3. Nous allons ajouter une méthode Iterable qui va être annotée par @RequestMapping qui définit le 'endpoint' ("/voitures") avec lequel la méthode va être mappée. Cette annotation permet de traiter toutes les méthodes HTTP (GET, POST, PUT, PATCH, DELETE, etc), il suffit de faire @RequestMapping("/voitures", method=GET) pour définir le endpoint 'voitures' qui va être manipulé par la méthode GET. Nous allons nous contenter maintenant de la méthode GET, qui est la méthode utilisée par défaut. On va injecter ensuite la classe VoitureRepo pour pouvoir utiliser sa méthode findAll() qui va nous retourner toutes les voitures de la Repository. Le code est comme suit :

```
@RestController
```

```

public class VoitureController {

    @Autowired

    private VoitureRepo voitureRepo;

    @RequestMapping("/voitures")

    public Iterable<Voiture> getVoitures(){

        return voitureRepo.findAll();

    }

}

```

4. Allez après sur Postman pour faire les tests. Choisir la méthode 'GET' et 'localhost:8080/voitures'. Appuyez sur 'Send' et vous allez remarquer qu'il y a un effet de boucle dû à la sérialisation répétée entre Voiture et Proprietaire. Pour stopper cet effet, nous allons ajouter l'annotation @JsonIgnore avant l'attribut de jointure dans les deux classes Voiture et Proprietaire, comme suit :

//Proprietaire.java

```
import com.fasterxml.jackson.annotation.JsonIgnore;
```

.....

```
@OneToMany(cascade = CascadeType.ALL, mappedBy = "proprietaire")
```

```
@JsonIgnore
```

```
private List<Voiture> voitures;
```

//-----

//Voiture.java

```
@JoinColumn(name = "proprietaire")
```

```
@NotNull
```

```
@JsonIgnore
```

```
private Proprietaire proprietaire;
```

5. Maintenant vous pouvez ré-exécuter et voir le résultat qui affiche la liste des voitures sur '**localhost:8080/voitures**', avec POSTMAN.
6. Voilà !!!